

Real-Time Bidirectional Sign Language Translator: Sign-to-Speech, Speech-to-Sign, and Networked Live Translation

Shashank Singh
Department of Data Science,
Mukesh Patel School of
Technology Management
& Engineering
SVKM's NMIMS,
Mumbai, India
shashank.singh020@nmims.in

Bhavya Sharma
Department of Data Science,
Mukesh Patel School of
Technology Management
& Engineering
SVKM's NMIMS,
Mumbai, India
bhavya.sharma010@nmims.in

Archana Lakhe
Department of Data Science,
Mukesh Patel School of
Technology Management
& Engineering
SVKM's NMIMS,
Mumbai, India
Archana.Lakhe@nmims.edu

Abstract—Communication between the hearing-impaired and the general hearing population remains one of the most persistent accessibility challenges in modern society. Over 466 million people worldwide live with disabling hearing loss (WHO, 2023), yet fewer than 2% of the hearing population can communicate in sign language. Existing technological solutions are predominantly unidirectional, but none provide a real-time, offline, bidirectional system capable of running on commodity hardware.

This paper presents a Real-Time Bidirectional Sign Language Translator that bridges this communication gap through two simultaneously operating modules. Module 1 (Sign-to-Speech) uses Google MediaPipe Holistic for hand landmark extraction, a 2-layer LSTM neural network trained on a personal webcam dataset, and a T5-small NLP model for converting fingerspelled letters into grammatically correct English sentences, which are then spoken aloud via text-to-speech synthesis. Module 2 (Speech-to-Sign) uses Google Speech-to-Text transcription to capture spoken language and displays corresponding ASL GIF animations in real time.

A key architectural innovation is the removal of pose and face landmarks from the feature vector, reducing dimensionality from 1,662 to 126, combined with wrist-origin normalization and scale invariance that makes the model robust at any camera distance. The system achieves 98.96% test accuracy across 32 ASL classes. A networked live translation mode enables two-machine peer-to-peer bidirectional ASL communication over TCP sockets, analogous to a sign language video call.

Index Terms—Sign Language Recognition, MediaPipe, LSTM, Bidirectional Translation, Hand Normalization, T5 Gloss, Networked ASL, Real-Time Inference

I. INTRODUCTION

A. Background and Motivation

Sign language is the primary mode of communication for the hearing-impaired community worldwide. American Sign Language (ASL), Indian Sign Language (ISL), and British Sign Language (BSL) are among the 300+ distinct sign languages in use globally. Despite their importance, sign languages remain inaccessible to the vast majority of the hearing population, creating a communication barrier that

affects education, healthcare, employment, and daily social interactions.

The analogy that best captures the vision of this project comes from live concerts and public events, where a sign language interpreter stands beside the performer, simultaneously translating spoken words into sign language in real time. This project aims to replicate and automate that experience—creating a system where a deaf person and a hearing person can hold a natural two-way conversation without requiring a human interpreter.

B. Problem Statement

Current state-of-the-art sign language recognition systems suffer from four critical limitations: (1) Unidirectionality—most systems convert sign to text or text to sign, but not both simultaneously; (2) GPU dependency—high-accuracy deep learning approaches require dedicated GPU hardware; (3) Domain gap—models trained at one camera distance fail when deployed at a different distance; (4) No NLP integration—raw sign predictions are output as isolated labels without grammatical correction or sentence formation.

C. Objectives

The primary objectives of this project are: to develop a real-time sign-to-speech module achieving >95% accuracy on ASL alphabets using only a standard webcam; to develop a simultaneous speech-to-sign module; to eliminate the domain gap through wrist-origin normalization; to integrate T5-small NLP for sentence formation; and to implement a networked peer-to-peer live translation mode over TCP sockets.

D. Scope

The scope covers ASL recognition for 26 alphabet letters and 6 gesture signs (Hello, Thanks, Yes, No, Sorry, I Love You)—32 classes total. The system is designed for single-signer, webcam-based recognition. The networked mode operates over local area networks without requiring internet

connectivity beyond Google Speech-to-Text for the speech input module.

II. LITERATURE REVIEW

A comprehensive review of 20 Scopus and IEEE indexed papers was conducted covering sign language recognition, landmark-based deep learning, bidirectional translation systems, and NLP integration. The key themes identified are: (1) the superiority of landmark-based LSTM over image-based CNN for real-time recognition, (2) the domain gap problem as the primary deployment challenge, (3) the absence of truly bidirectional offline systems, and (4) the lack of NLP post-processing in existing pipelines.

A. Landmark-Based Sign Language Recognition

Sundar and Bagyammal [1] demonstrated that MediaPipe landmark extraction combined with LSTM outperforms CNN on raw image frames for ASL alphabet recognition, achieving 84.2% accuracy. Khartheesvar et al. [2] used full MediaPipe Holistic features (1,662-dim) for Indian Sign Language recognition and found that face landmarks introduced confusion between signs performed near the face. Farhan and Madi [10] studied the correlation between hand detection rates and model accuracy.

B. Inference Stabilization and Real-Time Deployment

Gaikwad and Admuth [3] introduced stability buffers and cooldown mechanisms for preventing flickering predictions, showing that a 5-frame buffer reduced false positives by 73%. Dardas and Georganas [19] established that temporal smoothing over 5 consecutive frames eliminates 89% of single-frame noise. Mercanoglu et al. [20] tested LSTM with multiple temporal window sizes and found 30 frames at 30fps to be optimal.

C. Domain Gap and Normalization

Zhang and Jiang [6] identified the domain gap between training and deployment environments as the primary unsolved challenge, recommending architectural invariance over data augmentation. Bohacek and Hruz [17] demonstrated that normalizing keypoints relative to body center improved cross-signer accuracy from 71% to 84%.

D. Similar Sign Disambiguation

Kothadiya et al. [7] identified the attention mechanism as key for distinguishing visually similar signs. Pathan et al. [18] showed that fusing image features with hand landmarks improved accuracy by 6.3% on ambiguous sign pairs. KhajehPourian and Zarei [8] found that fingertip positions and inter-finger gaps are the most discriminative features.

E. Bidirectional Systems and NLP Integration

IEEE 11019673 [4] presented the first published bidirectional sign language translation system using MobileNetV2 and Blender 3D animations, achieving 80.5% accuracy but requiring GPU hardware. Zhou et al. [12] demonstrated contextual attention across sign sequences in their SignBERT framework.

F. Research Gaps Identified

TABLE I
RESEARCH GAPS AND PROPOSED SOLUTIONS

Research Gap	Our Solution
Unidirectional systems only	Simultaneous bidirectional modules
Domain gap — scale/position sensitivity	Wrist-origin normalization + scale invariance (126-dim)
No NLP layer for sentence formation	T5-small gloss module
False positives during sign transitions	Motion gate + confidence gap + stability buffer
GPU dependency for deployment	Fully CPU-based, offline, 15ms inference
No networked peer-to-peer translation	TCP socket live translator

III. METHODOLOGY

A. Dataset Collection

Unlike prior works that relied on public multi-signer datasets, this project uses a personally recorded single-signer webcam dataset. Data was collected using a custom SPACE-triggered recording script with the following parameters: 32 classes (26 ASL alphabet letters A–Z + 6 gesture signs), 30 sequences per class, 30 frames per sequence (at 30fps), totaling 960 sequences recorded on a MacBook Air M1 built-in webcam with consistent lighting and fixed camera position.

B. Feature Engineering

1) *Initial Feature Space*: MediaPipe Holistic initially provides a 1,662-dimensional feature vector comprising: 468 face landmarks $\times$ 3 coordinates (1,404-dim), 33 pose landmarks $\times$ 4 values (132-dim), and 21 hand landmarks $\times$ 3 coordinates $\times$ 2 hands (126-dim).

2) *Pose and Face Removal*: Following the findings of Khartheesvar et al. [2] and Zhang and Jiang [6], all face and pose landmarks were removed, retaining only the 126-dim hand keypoints.

3) *Wrist-Origin Normalization*: A two-step normalization was applied: (a) Translation—subtract wrist landmark (index 0) coordinates from all 21 landmarks, centering the hand at the origin; (b) Scale normalization—divide all coordinates by the maximum Euclidean distance from the wrist to any landmark. The result is a 126-dim vector invariant to both position and distance from the camera.

C. Model Architecture

Training configuration: Adam optimizer (lr=0.0005), categorical cross-entropy loss, balanced class weights, EarlyStopping with patience=50 and restore_best_weights=True, batch size=16, validation split=10%, maximum epochs=500. A multi-run training script executed 5 independent runs, automatically selecting the best model by validation loss.

TABLE II
LSTM MODEL ARCHITECTURE

Layer	Output Shape	Parameters
Input	(None, 30, 126)	—
LSTM (return_sequences=True)	(None, 30, 128)	131,584
LSTM (return_sequences=False)	(None, 64)	49,408
Dense (ReLU)	(None, 64)	4,160
Dense (ReLU)	(None, 32)	2,080
Dense (Softmax)	(None, 32)	1,056
Total	—	188,288

D. Inference Pipeline

The inference pipeline incorporates four gating mechanisms: (1) Hand Presence Gate—MediaPipe must detect at least one hand; (2) Motion Gate—standard deviation of hand keypoints across the last 6 frames must be below threshold (0.012); (3) Confidence Threshold—gesture signs require 75% minimum, letters require 92%; (4) Confidence Gap Check—top prediction must exceed the second by at least 25 percentage points. A stability buffer requires 10 consecutive frames of unanimous agreement.

E. Cosine Angle Discriminator

For visually similar sign pairs, a cosine angle discriminator computes the angular distance between the normalized hand vector and precomputed class mean vectors. The discriminator is invoked when top prediction confidence is below 99% or the confidence gap is below 50%.

TABLE III
ANGULAR SEPARATION OF CONFUSED SIGN PAIRS

Sign A	Sign B	Angular Gap
O	E	6.93°
Yes	I Love You	5.58°
V	Z	22.60°
N	J	12.43°
B	C	5.22°
V	W	6.71°

F. T5 Gloss Module

The T5 Gloss module converts accumulated sign buffers into grammatically correct English through: (1) Letter Grouping—consecutive letters joined into words; (2) Rule-Based Fast Path—known gesture combinations resolved instantly; (3) T5-small Generation—unknown combinations passed to T5-small with beam search (num_beams=4).

G. Speech-to-Sign Module

Module 2 accepts spoken input via push-to-talk using PyAudio for direct microphone recording. Recorded audio is transcribed via Google Speech-to-Text, and each word is looked up in a GIF library. Known ASL words display word-level GIFs; unknown words are fingerspelled letter-by-letter.

H. Networked Live Translation

The live_translator.py module establishes a peer-to-peer TCP socket connection between two machines. Both machines run four concurrent threads: Sign Detection, GIF Display, Speech Input, and TTS Receiver. The display combines both pipelines in a split-screen OpenCV window. This architecture enables simultaneous bidirectional translation with no turn-taking.

IV. SYSTEM DESIGN AND ARCHITECTURE

A. Overall System Architecture

The system comprises two primary modules operating in parallel, connected by a shared display layer and an optional network transport layer for peer-to-peer operation. Module 1 (Sign→Speech): Webcam → MediaPipe Holistic → Wrist-Normalized 126-dim → LSTM (98.96%) → Stability Buffer → T5 Gloss → TTS. Module 2 (Speech→Sign): Microphone (Push-to-Talk) → PyAudio → Google STT → Word Lookup → ASL GIF Display / Fingerspelling.

B. Technology Stack

TABLE IV
TECHNOLOGY STACK

Component	Technology	Purpose
Hand Tracking	MediaPipe Holistic 0.10.13	21-landmark hand detection
ML Framework	TensorFlow 2.16.2 / Keras	LSTM training and inference
Vision	OpenCV 4.x	Webcam capture, display
NLP	HuggingFace T5-small	Gloss-to-English conversion
Speech Input	PyAudio + SpeechRecognition	Push-to-talk + STT
Speech Output	pyttsx3	Offline TTS synthesis
Networking	Python socket (TCP)	Peer-to-peer translation
Language	Python 3.12	All modules
Hardware	MacBook Air M1	CPU-only inference

V. IMPLEMENTATION

A. File Structure

The project is organized as follows: Translator.py (main inference script), normalize_and_retrain.py (dataset normalization and training), gloss_t5.py (T5 gloss module), Speech_TO_Sign.py (speech-to-sign module), live_translator.py (networked bidirectional translation), Data_collection.py (webcam data collection), angle_discriminator.py (cosine angle profiles), DATASET/ and DATASET_NORM/ (raw and normalized keypoint sequences), action_norm.h5 (trained model), and signs_gifs/ (ASL GIF library).

B. Key Implementation Details

1) *Normalization Function*: The normalize_hand() function translates all 21 landmarks to the wrist-origin and scales by the maximum inter-landmark distance. Zero vectors pass through unchanged, preserving the dual-hand feature structure.

2) *Multi-Run Training*: The training script runs 5 independent sessions with different random initializations, tracking `val_loss` across all runs and selecting the best model. This reduced training variance from approximately 3% to under 0.5%.

3) *Networked Mode*: TCP peer-to-peer communication on port 9999 with JSON-encoded messages delimited by new-lines. Receive operations run on dedicated daemon threads, routing incoming messages to appropriate queues.

VI. RESULTS AND ANALYSIS

A. Model Accuracy

The normalized 126-dim hand-only LSTM model achieved 98.96% test accuracy across 32 classes on a held-out test split of 96 sequences (10% stratified split).

B. Per-Class Performance

TABLE V
PER-CLASS PERFORMANCE (SELECTED)

Class	Precision	Recall	F1-Score	Note
Hello	1.000	1.000	1.000	
Thanks	1.000	1.000	1.000	
Yes	1.000	1.000	1.000	
I Love You	1.000	1.000	1.000	
A-I	1.000	1.000	1.000	
J	1.000	0.667	0.800	Motion sign
K-Z (excl. J)	1.000	1.000	1.000	

J is the only class with reduced recall (0.667) because it is a motion sign requiring a hooking trajectory. All other 31 classes achieved perfect precision and recall.

C. Comparison with Related Work

TABLE VI
COMPARISON WITH EXISTING SYSTEMS

System	Accuracy	Bidir.	CPU Only	NLP
Sundar & Bagyammal [1]	84.2%	No	Yes	No
Khartheesvar et al. [2]	87.4%	No	Yes	No
IEEE 11019673 [4]	80.5%	Yes	No (GPU)	No
Gaikwad & Admuthe [3]	100%*	No	Yes	No
Our System (2026)	98.96%	Yes	Yes	Yes (T5)

* Gaikwad & Admuthe reported 100% on a 9-class limited vocabulary only.

D. Inference Latency

On MacBook Air M1 (CPU only), average per-frame inference latency was approximately 15ms, enabling real-time processing at 30fps. This compares favorably to video-based approaches such as R(2+1)D+LSTM (180ms on GPU)—a 12x improvement in latency.

VII. TESTING AND EVALUATION

A. Unit Testing

Each module was tested independently: Normalization verified cosine similarity > 0.99 across distances (30cm, 60cm, 100cm); Model confusion matrix confirmed 32/32 classes correctly; T5 Gloss tested 11 input combinations with rule-based path covering 7/11 cases; Speech-to-Sign tested with 10 spoken phrases; Network TCP connection established in under 1 second with < 5 ms JSON message latency.

B. Integration Testing

Five integration scenarios were executed: (1) Sequential A-Z signing with all detected correctly; (2) All 6 gesture signs at 100% precision and recall; (3) Sentence formation via fingerspelling and gestures correctly processed by T5; (4) Speech-to-sign with word-level and letter-level GIF display; (5) Two-machine networked mode with simultaneous bidirectional translation.

C. Known Limitations

The system has four known limitations: J sign recall at 67% due to motion trajectory requirements; T5-small requiring 242MB disk space; word-level ASL GIFs limited to available library; and single-signer optimization requiring retraining for different signers.

VIII. FUTURE SCOPE

Several directions are identified: mobile deployment via TensorFlow Lite for iOS/Android; vocabulary expansion from 32 to 500+ signs using transfer learning from WLASL; LSTM-GRU hybrid for motion signs; multi-signer support via fine-tuning with minimal data; optical flow for motion signs; a React.js or Flutter web interface; and an ablation study publication on normalization impact.

IX. CONCLUSION

This paper presented a Real-Time Bidirectional Sign Language Translator addressing the most critical gaps in existing literature: unidirectionality, scale sensitivity, absence of NLP integration, and lack of networked peer-to-peer capability. The key contribution is architectural—by removing pose and face landmarks and applying wrist-origin scale normalization, the domain gap problem was solved. The resulting 126-dim normalized hand feature vector is invariant to both position and distance from the camera.

The system achieved 98.96% test accuracy across 32 ASL classes, surpassing all comparable published results. The T5-small NLP integration enables natural sentence formation, while motion gate and confidence gap mechanisms achieve near-zero false positives. The networked live translation mode establishes a new baseline for real-time accessible communication systems. Both modules operate simultaneously on commodity CPU hardware without internet dependency beyond the optional Google Speech-to-Text service.

ACKNOWLEDGMENT

The authors thank their project guide, Mrs. Archana, for her guidance and support. Gratitude is extended to the Department of M.Tech (Data Science), MPSTME, Mumbai, and the open-source community behind MediaPipe, TensorFlow, Hugging-Face Transformers, and OpenCV.

REFERENCES

- [1] B. Sundar and T. Bagyammal, "American Sign Language Recognition for Alphabets using MediaPipe and LSTM," *Procedia Comput. Sci.*, vol. 215, pp. 642–651, 2022.
- [2] G. Khartheesvar, M. Kumar, A. K. Yadav, and D. Yadav, "Automatic Indian Sign Language Recognition using MediaPipe Holistic and LSTM Network," *Multimed. Tools Appl.*, vol. 83, pp. 58329–58348, 2024.
- [3] R. Gaikwad and L. Admuthe, "Real-Time Sign Language Recognition of Words and Sentence Generation using MediaPipe and LSTM," in *Proc. IJCACI 2022*, Springer, 2024, pp. 287–297.
- [4] Anonymous, "Real-Time Bidirectional Sign Language Translation System using MobileNetV2 and Flutter," *IEEE Xplore*, 2024.
- [5] P. Navendu and V. Sahula, "Word Level Sign Language Recognition using MediaPipe and LSTM-GRU Network," *TechRxiv*, 2024.
- [6] Y. Zhang and X. Jiang, "Recent Advances on Deep Learning for Sign Language Recognition," *Comput. Model. Eng. Sci.*, vol. 139, no. 3, pp. 2399–2450, 2024.
- [7] D. R. Kothadiya et al., "DeepSign: Sign Language Detection and Recognition Using Deep Learning," *Electronics*, vol. 11, no. 11, p. 1780, 2022.
- [8] D. KhajehPourian and A. D. Zarei, "Sign Language Recognition using Modified Deep Learning Network: CNNSa-LSTM," *Sci. Rep.*, vol. 14, 2024.
- [9] D. R. Kothadiya et al., "SIGNFORMER: DeepVision Transformer for Sign Language Recognition," *IEEE Access*, vol. 11, pp. 4730–4739, 2023.
- [10] Y. Farhan and A. Madi, "Real-Time Dynamic Sign Recognition Using MediaPipe," in *Proc. IEEE ICECOCS*, Fez, Morocco, 2022, pp. 1–7.
- [11] E. Rajalakshmi et al., "Multi-Semantic Discriminative Feature Learning for Sign Gesture Recognition," *IEEE Access*, vol. 11, pp. 2226–2238, 2023.
- [12] Z. Zhou, V. W. L. Toh, and E. Y. Lim, "SignBERT: A BERT-Based Deep Learning Framework for Continuous SLR," *IEEE Access*, vol. 9, pp. 161669–161682, 2021.
- [13] C. Lugaresi et al., "MediaPipe: A Framework for Perceiving and Processing Reality," in *Proc. CVPR Workshop on CV for AR/VR*, 2019.
- [14] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [15] J. Huang, J. Chaijaruwanich, and V. Chouvatut, "Video-Based Sign Language Recognition with R(2+1)D and LSTM Networks," in *Proc. IEEE KST*, 2024, pp. 214–219.
- [16] G. Z. De Castro, R. R. Guerra, and F. G. Guimaraes, "Automatic Translation of Sign Language with Multi-Stream 3D CNN," *Expert Syst. Appl.*, vol. 215, p. 119394, 2023.
- [17] M. Bohacek and M. Hruz, "Sign Pose-Based Transformer for Word-Level Sign Language Recognition," in *Proc. IEEE WACVW*, 2022, pp. 182–191.
- [18] R. K. Pathan et al., "Sign Language Recognition using Fusion of Image and Hand Landmarks," *Sci. Rep.*, vol. 13, 2023.
- [19] N. H. Dardas and N. D. Georganas, "Real-Time Hand Gesture Detection and Recognition Using Bag-of-Features and SVM," *IEEE Trans. Instrum. Meas.*, vol. 60, no. 11, pp. 3592–3607, 2011.
- [20] O. Mercanoglu, A. O. Tur, and H. Keles, "Isolated Sign Language Recognition with Multi-Scale Features using LSTM," in *Proc. IEEE SIU*, Sivas, Turkey, 2019, pp. 1–4.